Sumit Pandey
2024010110
Group 5

1. Write a program to implement producer consumer scenario using POSIX shared memory.

```
#include <iostream>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <semaphore.h>
#include <unistd.h>
#include <cstring>

const char* sharedMemoryName = "/mySharedMemory";
const char* semEmptyName = "/semEmpty";
const char* semFullName = "/semFull";
const char* semMutexName = "/semMutex";
const int bufferSize = 10;

struct SharedMemory {
   int buffer[bufferSize];
   int in, out;
};

void producer() {
   int shm_fd = shm_open(sharedMemoryName, O_CREAT | O_RDWR, 0666);
   ftruncate(shm_fd, sizeof(SharedMemory));
   SharedMemory* sharedMemory = (SharedMemory*)mmap(0, sizeof(SharedMemory), PROT_WRITE,
MAP_SHARED, shm_fd, 0);

   sem_t* semEmpty = sem_open(semEmptyName, O_CREAT, 0666, bufferSize);
   sem_t* semFull = sem_open(semFullName, O_CREAT, 0666, 0);
   sem_t* semMutex = sem_open(semMutexName, O_CREAT, 0666, 1);

   for (int i = 0; i < 20; ++i) {
      sem_wait(semEmpty);
      sem_wait(semMutex);

      sharedMemory->buffer[sharedMemory->in] = i;
```

```
        sharedMemory->in = (sharedMemory->in + 1) % bufferSize;
        std::cout << "Produced: " << i << std::endl;

        sem_post(semMutex);
        sem_post(semFull);

        sleep(1);
    }

    sem_close(semEmpty);
    sem_close(semFull);
    sem_close(semMutex);
    munmap(sharedMemory, sizeof(SharedMemory));
    close(shm_fd);
}

void consumer() {
    int shm_fd = shm_open(sharedMemoryName, O_RDONLY, 0666);
    SharedMemory* sharedMemory = (SharedMemory*)mmap(0, sizeof(SharedMemory), PROT_READ, MAP_SHARED,
shm_fd, 0);

    sem_t* semEmpty = sem_open(semEmptyName, 0);
    sem_t* semFull = sem_open(semFullName, 0);
    sem_t* semMutex = sem_open(semMutexName, 0);

    for (int i = 0; i < 20; ++i) {
        sem_wait(semFull);
        sem_wait(semMutex);

        int item = sharedMemory->buffer[sharedMemory->out];
        sharedMemory->out = (sharedMemory->out + 1) % bufferSize;
        std::cout << "Consumed: " << item << std::endl;

        sem_post(semMutex);
        sem_post(semEmpty);
```

```
        sleep(2);
    }

    sem_close(semEmpty);
    sem_close(semFull);
    sem_close(semMutex);
    munmap(sharedMemory, sizeof(SharedMemory));
    close(shm_fd);
}

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        producer();
    } else {
        sleep(1); // Ensure producer
    }
}
```

```
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab4$ g++ -o 1 1.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab4$ ./1
Produced: 0
Produced: 1
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab4$ Produced: 2
Produced: 3
Produced: 4
Produced: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9

ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab4$
```

2. Write a program to implement inter process communication between the parent process and the child process using ordinary Pipes.

```
#include <iostream>
#include <unistd.h>
#include <cstring>
```

```
using namespace std;

int main() {
    int pipefd[2];
    pid_t pid;
    const char* message = "Hello from parent to child!";
    char buffer[256];

    // Create pipe
    if (pipe(pipefd) == -1) {
        cerr << "Pipe failed" << endl;
        return 1;
    }

    // Fork a child process
    pid = fork();
    if (pid < 0) {
        cerr << "Fork failed" << endl;
        return 1;
    }

    if (pid > 0) {
        // Parent process
        close(pipefd[0]); // Close unused read end

        // Write to pipe
        write(pipefd[1], message, strlen(message) + 1);
        close(pipefd[1]); // Close write end after writing

        cout << "Parent sent message: " << message << endl;
    } else {
        // Child process
        close(pipefd[1]); // Close unused write end
```

```
    // Read from pipe
    read(pipefd[0], buffer, sizeof(buffer));
    close(pipefd[0]); // Close read end after reading

    cout << "Child received message: " << buffer << endl;
  }

  return 0;
}
```

```
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab4$ ./2
Parent sent message: Hello from parent to child!
Child received message: Hello from parent to child!
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab4$
```

3. Write program to implement IPC through message queues.

```
//Producer
#include <iostream>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <cstring>
using namespace std;

struct message_buffer {
  long message_type;
  char message_text[100];
};

int main() {
  key_t key;
  int msgid;

  // Generate a unique key
  key = ftok("progfile", 65);

  // Create a message queue and return identifier
```

```
    msgid = msgget(key, 0666 | IPC_CREAT);

    message_buffer message;
    message.message_type = 1;

    cout << "Enter message to send: ";
    cin.getline(message.message_text, 100);

    // Send message
    msgsnd(msgid, &message, sizeof(message), 0);

    cout << "Message sent: " << message.message_text << endl;

    return 0;
}



//Consumer
#include <iostream>
#include <sys/ipc.h>
#include <sys/msg.h>
using namespace std;

struct message_buffer {
    long message_type;
    char message_text[100];
};

int main() {
    key_t key;
    int msgid;

    // Generate a unique key
    key = ftok("progfile", 65);
```

```
    // Create a message queue and return identifier
    msgid = msgget(key, 0666 | IPC_CREAT);

    message_buffer message;

    // Receive message
    msgrcv(msgid, &message, sizeof(message), 1, 0);

    cout << "Message received: " << message.message_text << endl;

    // To destroy the message queue
    msgctl(msgid, IPC_RMID, NULL);

    return 0;
}
```

```
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab4$ ./3_1
Enter message to send: Hello
Message sent: Hello
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab4$ lsof | grep 3_2.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab4$ g++ -o 3_2 3_2.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab4$ ./3_2
Message received: Hello
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab4$
```